Feature detectors

2026-03-13

SDU

Video stabilization

Data from sky

DataFromSky & COWI – traffic analysis in Copenhagen

A moving camera

2017_06_22_1301 Roersvej støder op til Vesterbro

Data flow

The flow of data for analyzing traffic videos are often structured as follows:

- Stabilize video
  - Today
- Background subtraction
  - Next week
- Track moving objects
  - Kalman filters - not covered in this course

Video stabilization

- Feature detection
  - SIFT
  - AKAZE
  - ORB
- Image alignment
  - 2D to 2D image transform

2D planar transformations

Properties of 2D planer transformation

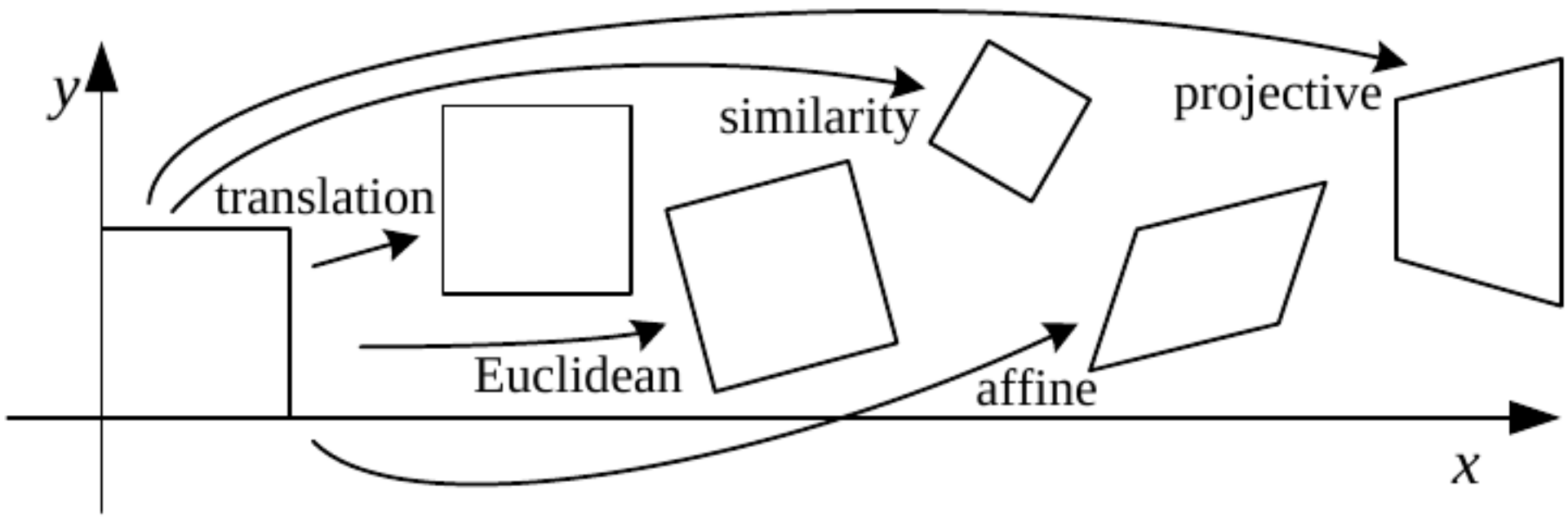
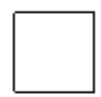
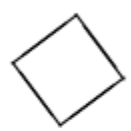
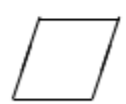


Table of properties

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \boldsymbol{I} \mid \boldsymbol{t} \end{bmatrix}_{2\times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \boldsymbol{R} \mid \boldsymbol{t} \end{bmatrix}_{2\times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\boldsymbol{R} \mid \boldsymbol{t} \end{bmatrix}_{2\times 3}$	4	angles	
affine	$\begin{bmatrix} \boldsymbol{A} \end{bmatrix}_{2\times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\boldsymbol{H}} \end{bmatrix}_{3\times 3}$	8	straight lines	

Translation

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Rotation and translation

$$x' = \cos(\theta) \cdot x + \sin(\theta) \cdot y + \Delta x$$
$$y' = \sin(\theta) \cdot x + \cos(\theta) \cdot y + \Delta y$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & \Delta x \\ \sin(\theta) & \cos(\theta) & \Delta y \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Similarity transform

$$x' = s\cos(\theta) \cdot x + s\sin(\theta) \cdot y + \Delta x$$
$$y' = s\sin(\theta) \cdot x + s\cos(\theta) \cdot y + \Delta y$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s\cos(\theta) & -s\sin(\theta) & \Delta x \\ s\sin(\theta) & s\cos(\theta) & \Delta y \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Affine transform

$$x' = a_{00} \cdot x + a_{01} \cdot y + a_{02}$$
$$y' = a_{10} \cdot x + a_{11} \cdot y + a_{12}$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Perspective transformation

$$x' = \frac{h_{00} \cdot x + h_{01} \cdot y + h_{02}}{h_{20} \cdot x + h_{21} \cdot y + h_{22}}$$

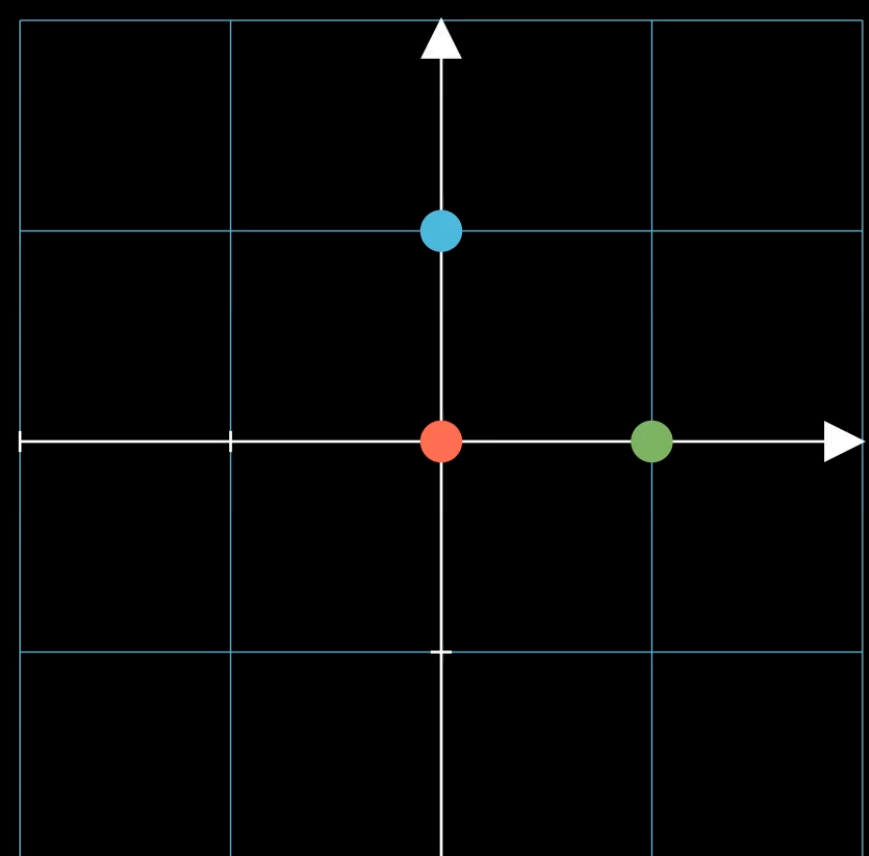
$$y' = \frac{h_{10} \cdot x + h_{11} \cdot y + h_{12}}{h_{20} \cdot x + h_{21} \cdot y + h_{22}}$$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Homogeneous coordinates

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \sim \begin{pmatrix} s \cdot u \\ s \cdot v \\ s \end{pmatrix}$$

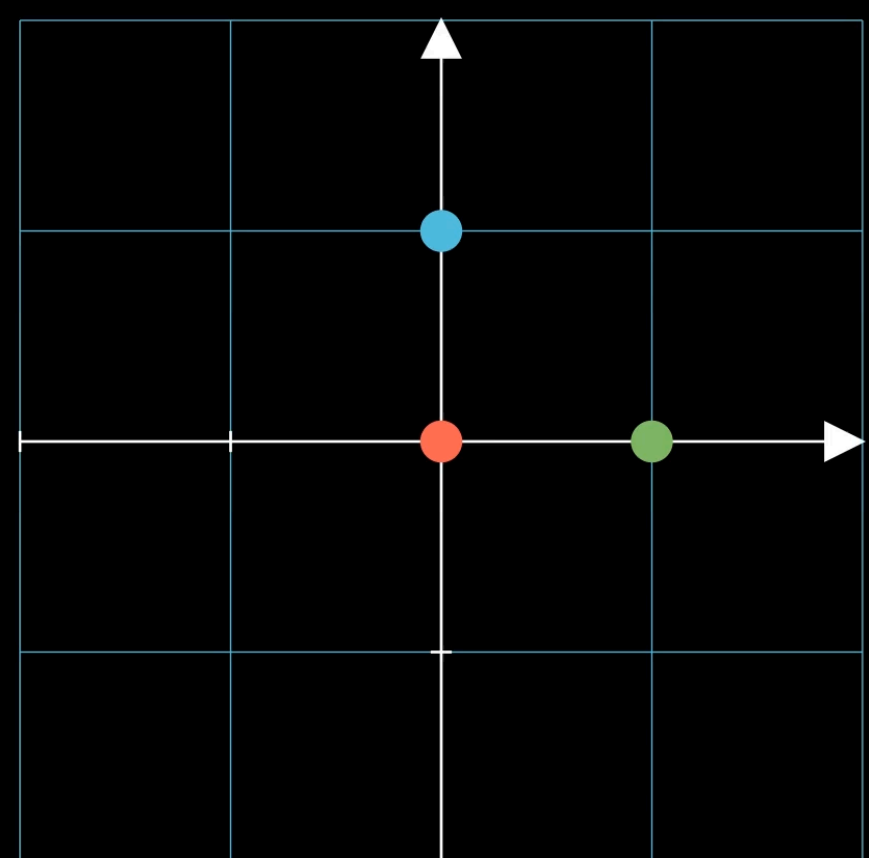
Animation - translation



Translation

$$\begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

Animation - rotation



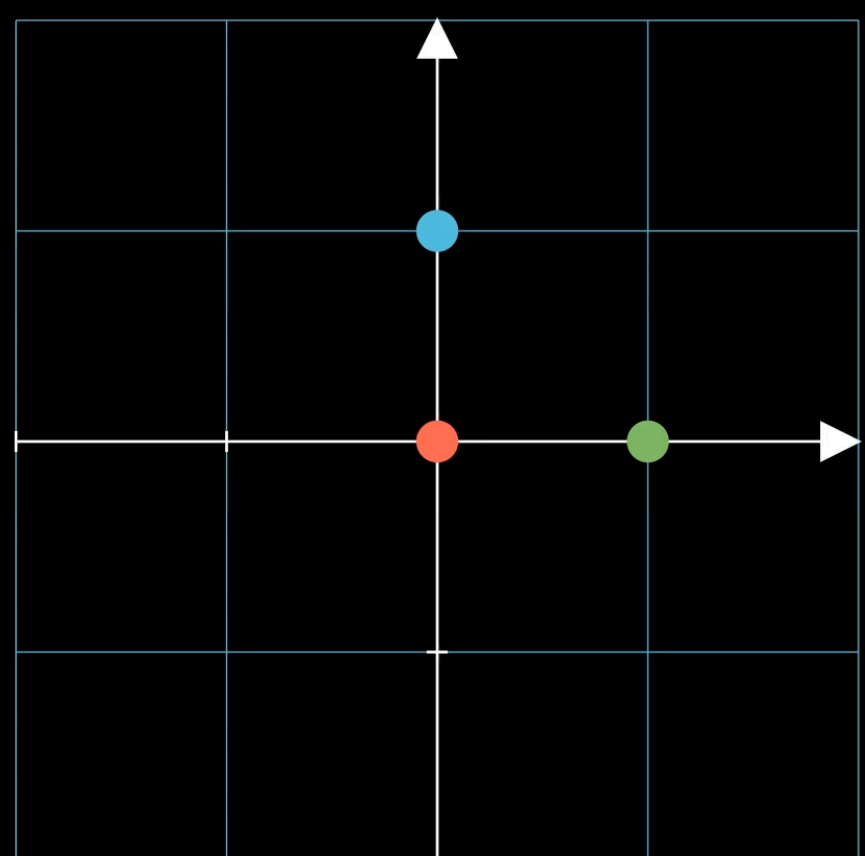
Rotation

$$\begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

SDU

SDU

Animation - scaling



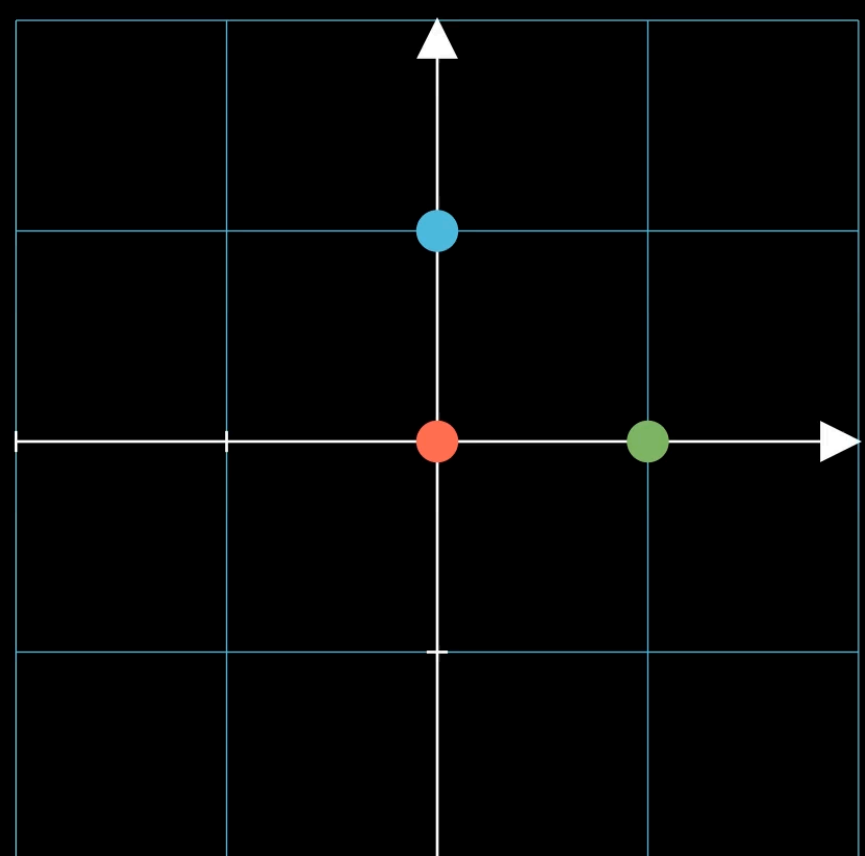
Scaling

$$\begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

SDU

SDU

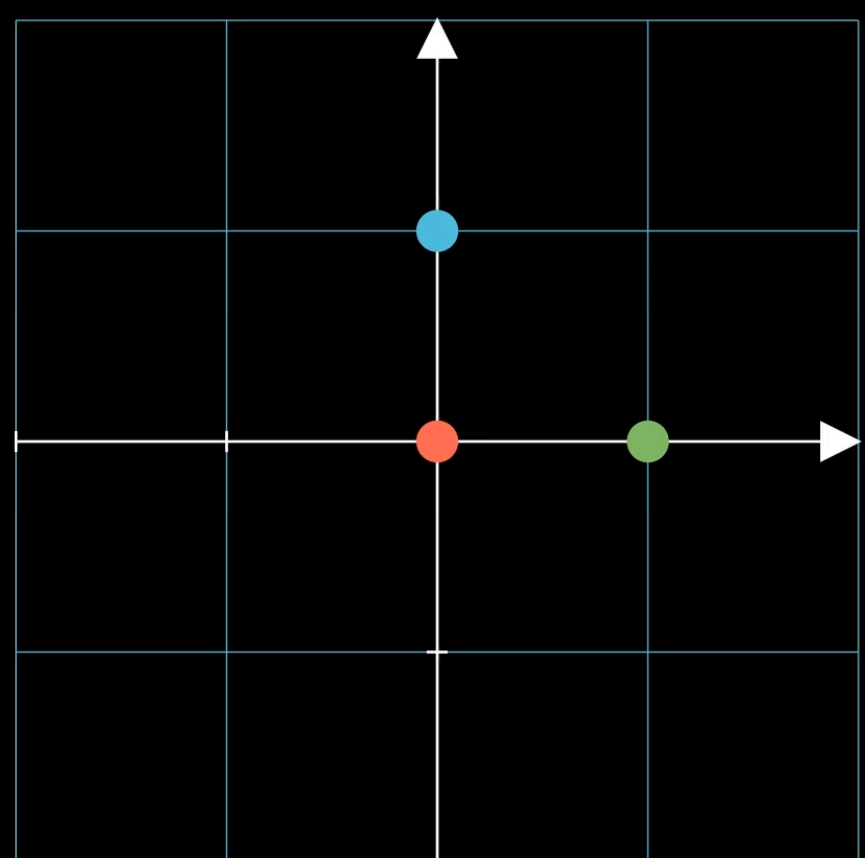
Animation - affine



Affine

$$\begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

Animation - perspective



Perspective

$$\begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

SDU

SDU

Matrix representation

All 2D to 2D image transforms can be written in the form

$$\begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix} = H \cdot \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$$

where H is a 3×3 matrix.

How to determine H?

Set up enough equations to solve for all the unknowns.

Determining an affine transform from three point pairs

$$x' = a_{11} \cdot x + a_{12} \cdot y + a_{13}$$
$$y' = a_{21} \cdot x + a_{22} \cdot y + a_{23}$$

What are the unknowns?

$a_{11}, a_{12}, a_{13}, a_{21}, a_{22}$ and a_{23}.

Set up equations from a point pair

$$x \cdot a_{11} + y \cdot a_{12} + 1 \cdot a_{13} + 0 \cdot a_{21} + 0 \cdot a_{22} + 0 \cdot a_{23} = x'$$
$$0 \cdot a_{11} + 0 \cdot a_{12} + 0 \cdot a_{13} + x \cdot a_{21} + y \cdot a_{22} + 1 \cdot a_{23} = y'$$

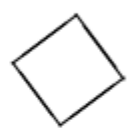
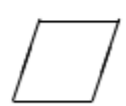
$$\begin{pmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{pmatrix} \cdot \begin{pmatrix} a_{11} \\ a_{12} \\ a_{13} \\ a_{21} \\ a_{22} \\ a_{23} \end{pmatrix} = \begin{pmatrix} x' \\ y' \end{pmatrix}$$

Can this be solved?

More equations

$$\begin{pmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \\ x_3 & y_3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_3 & y_3 & 1 \end{pmatrix} \cdot \begin{pmatrix} a_{11} \\ a_{12} \\ a_{13} \\ a_{21} \\ a_{22} \\ a_{23} \end{pmatrix} = \begin{pmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ x'_3 \\ y'_3 \end{pmatrix}$$

Transformation properties

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \boldsymbol{I} \mid \boldsymbol{t} \end{bmatrix}_{2\times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \boldsymbol{R} \mid \boldsymbol{t} \end{bmatrix}_{2\times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\boldsymbol{R} \mid \boldsymbol{t} \end{bmatrix}_{2\times 3}$	4	angles	
affine	$\begin{bmatrix} \boldsymbol{A} \end{bmatrix}_{2\times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\boldsymbol{H}} \end{bmatrix}_{3\times 3}$	8	straight lines	

The SIFT feature detector

Introduction & Motivation

- Why local features?
  - Stable landmarks for matching between images
  - Useful for: SLAM, mapping, 3D reconstruction
- Key requirements for a good feature:
  - Repeatable
  - Distinctive
  - Robust to scale, rotation, illumination

What is SIFT?

- Introduced by David Lowe (1999, 2004)
- Two main components:
  - <u>Feature detector</u>
  - <u>Feature descriptor</u>
- Designed to be:
  - Scale invariant
  - Rotation invariant
  - Robust to noise and moderate viewpoint changes

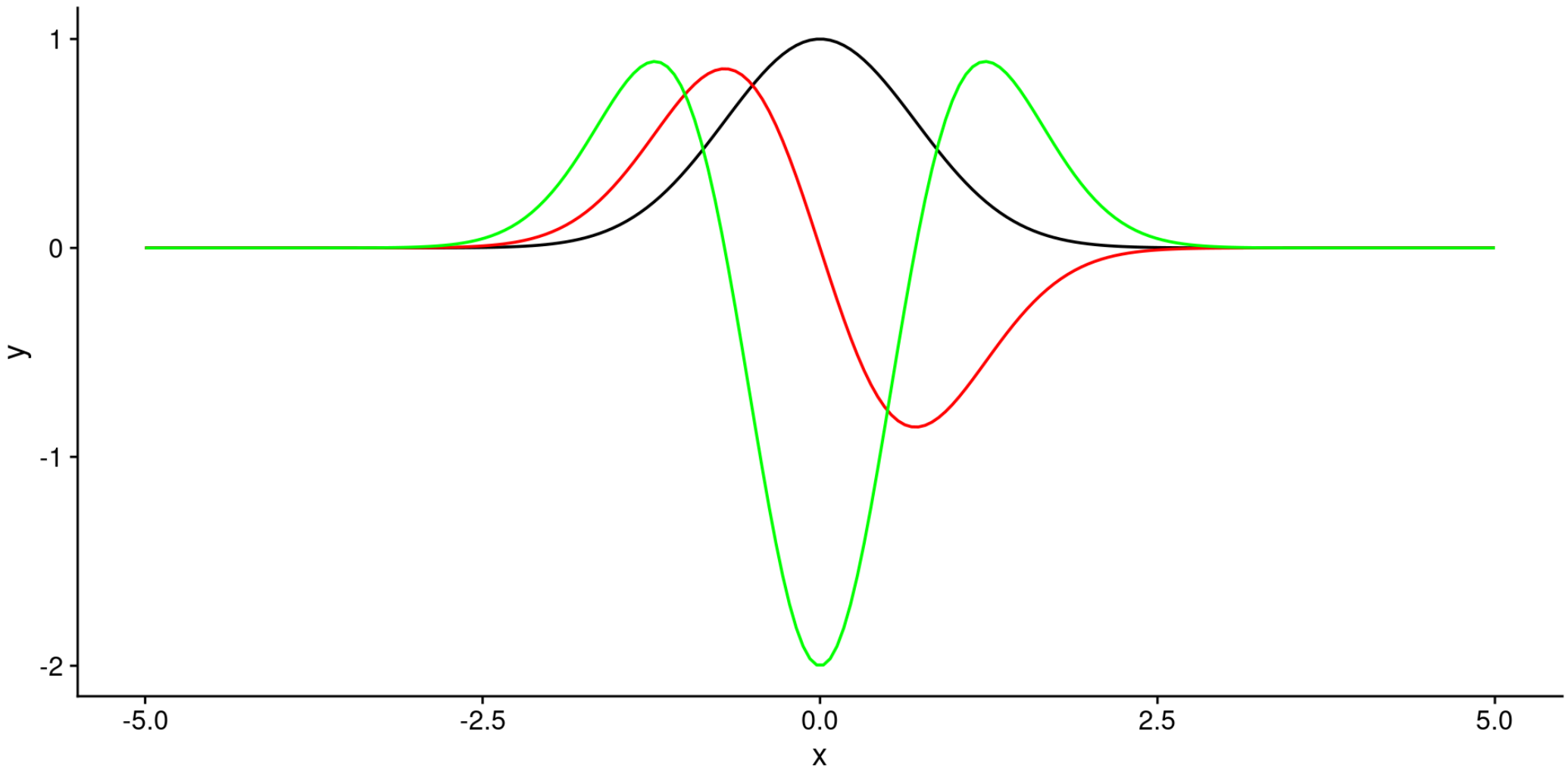
Part I — Feature Detection

Laplacian of Gaussian (LoG)

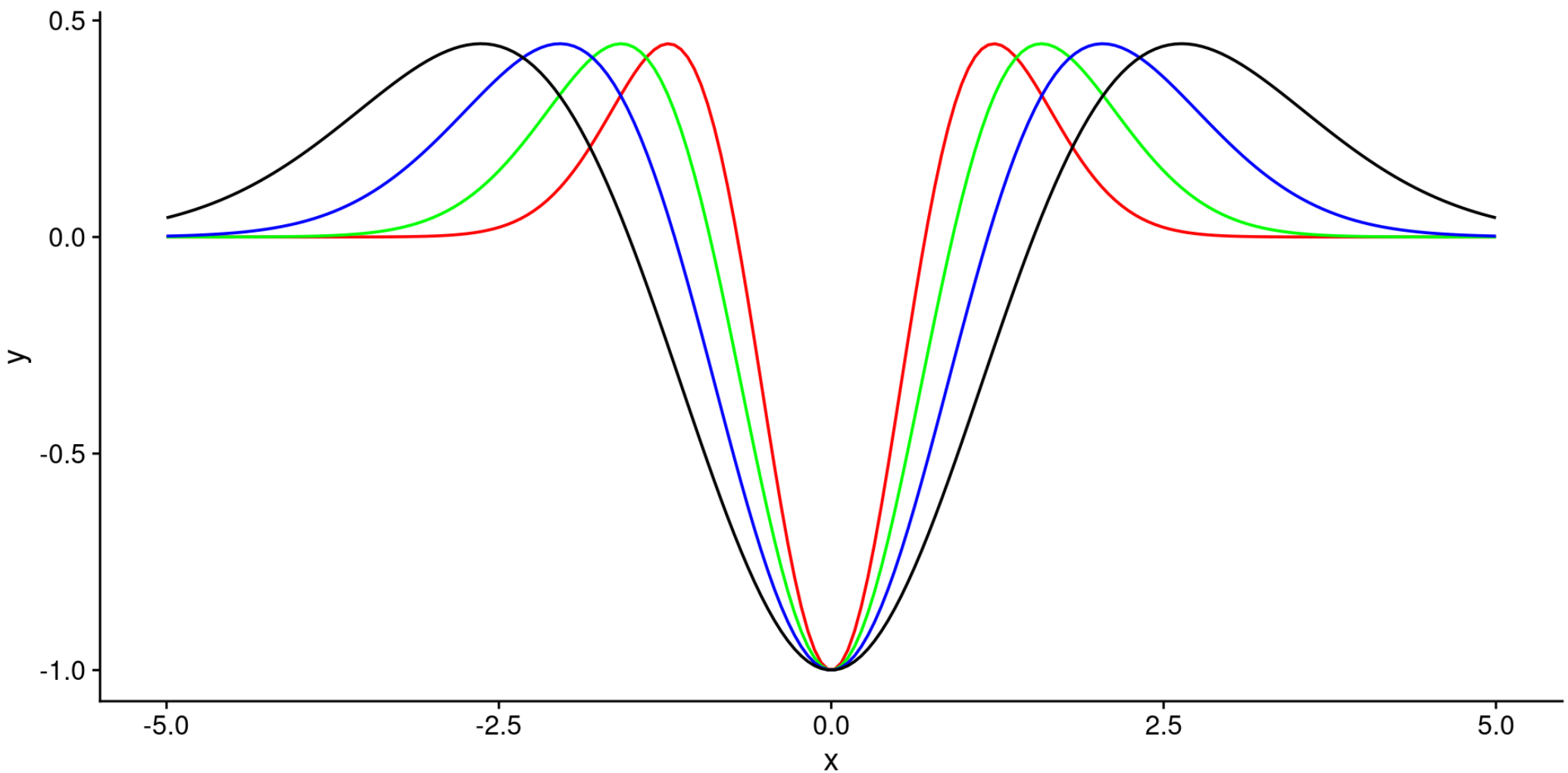
$$\frac{d}{dx}e^{-x^2} = -2x \cdot e^{-x^2}$$

$$\frac{d}{dx}\left(-2 \cdot x \cdot e^{-x^2}\right) = 4x^2 \cdot e^{-x^2} - 2 \cdot e^{-x^2}$$

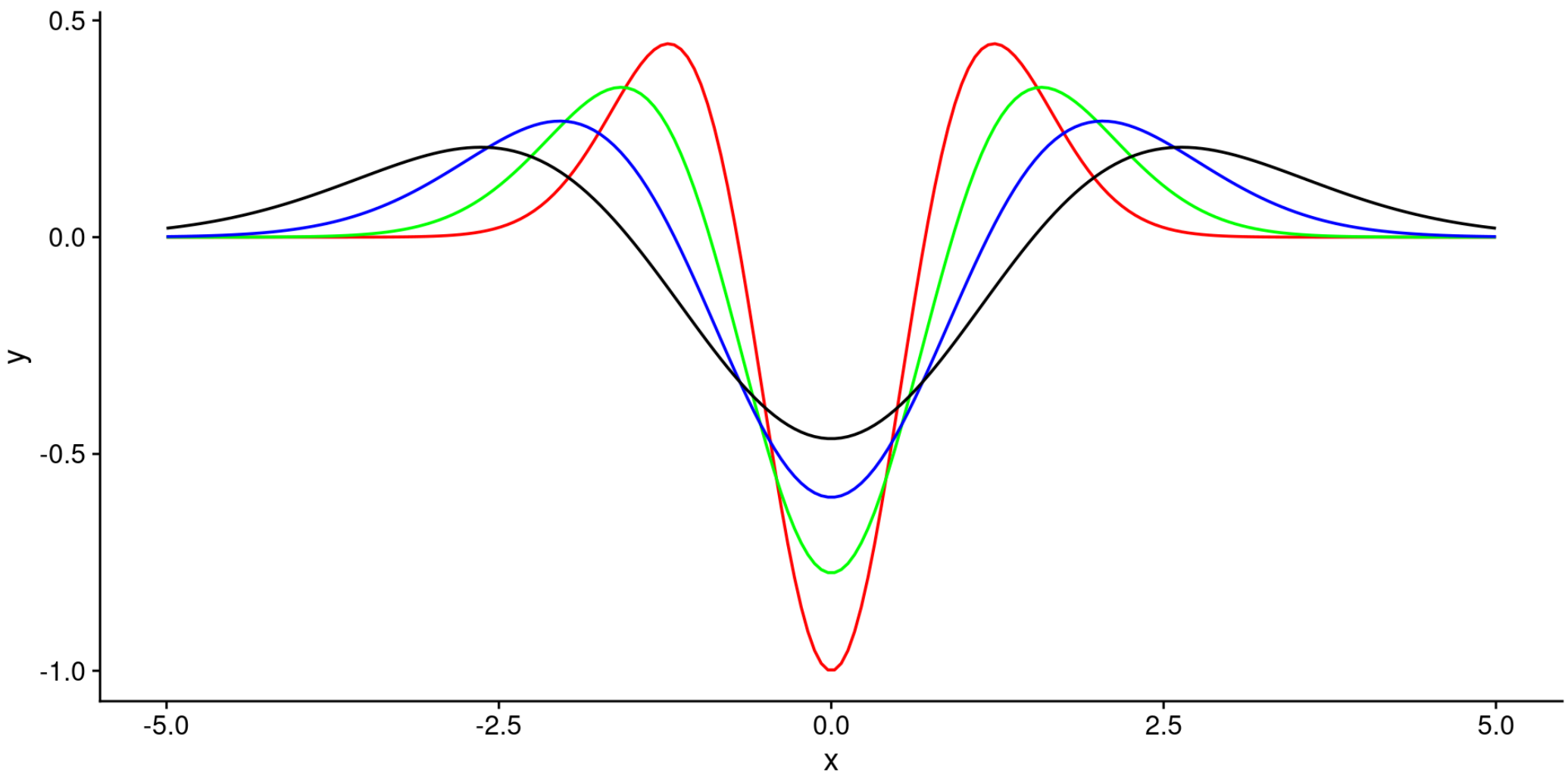
Mexican hat



Scales



Better scaling



Animation

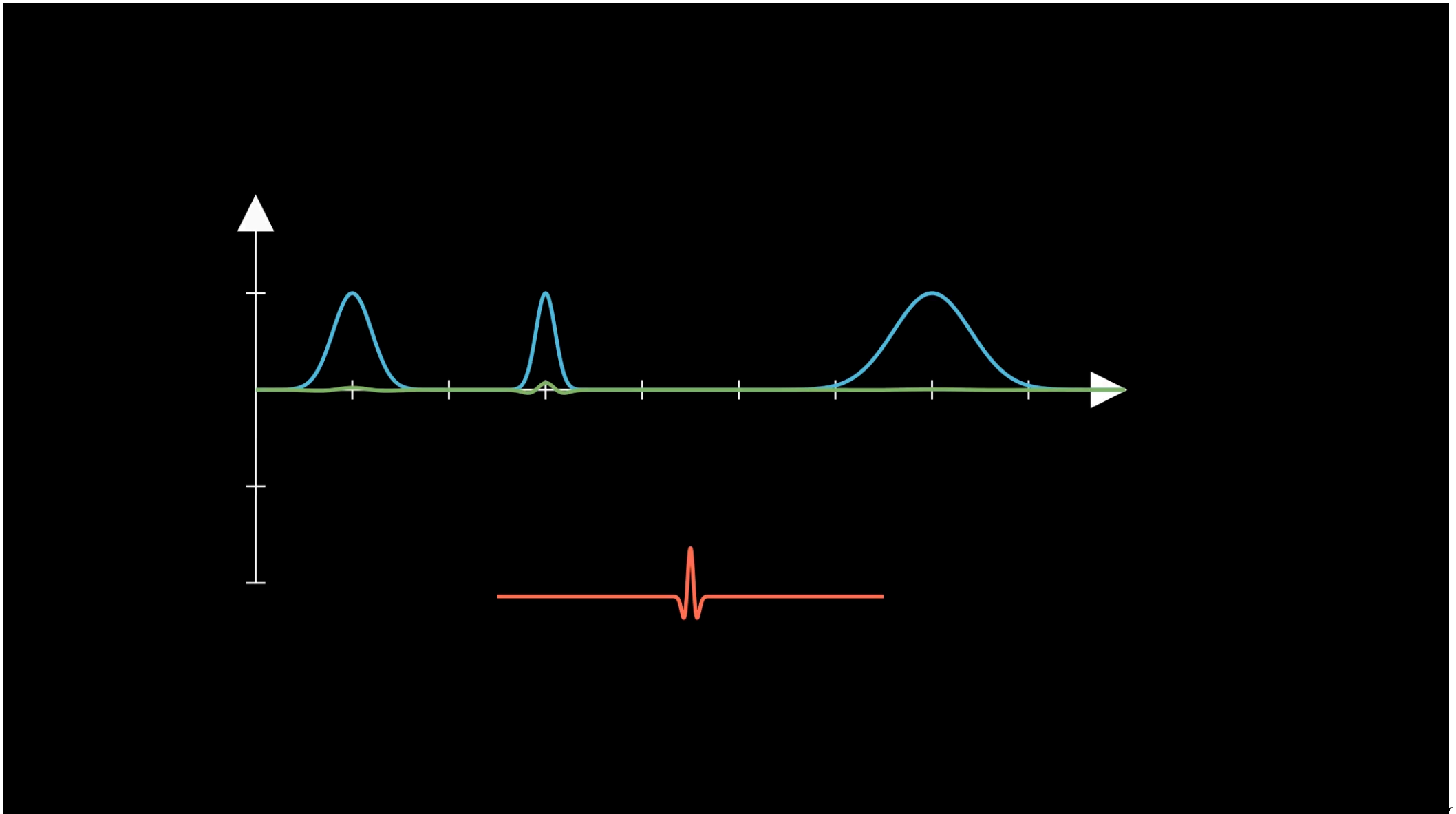


Image stack

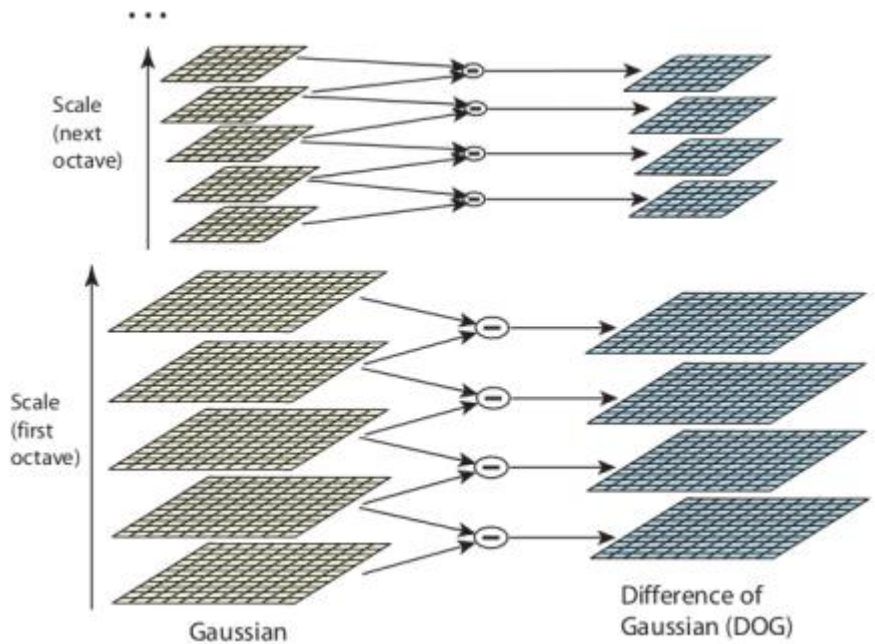
Look at these images

`materials/experiments/feature_detection/output/`

Scale-Space Construction

- Goal: detect keypoints at different image scales
- Gaussian blur used to generate scale-space
- DoG (Difference-of-Gaussians):
  - Approximates Laplacian of Gaussian (LoG)
  - Efficient to compute

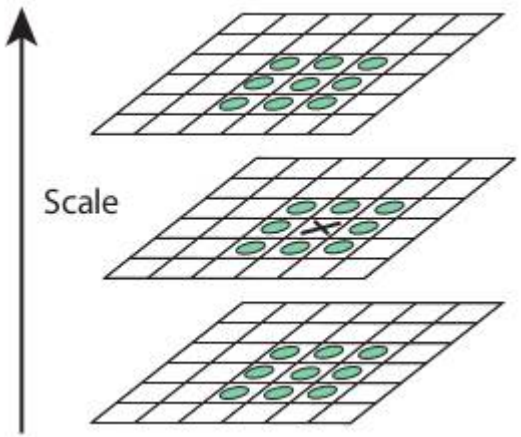
Scale space pyramid



DoG Extrema Detection

- Each pixel tested against 26 neighbors:
  - 8 in same scale
  - 9 above
  - 9 below
- Local maxima/minima ⇒ candidate keypoints

Extrema detection



Keypoint Localization

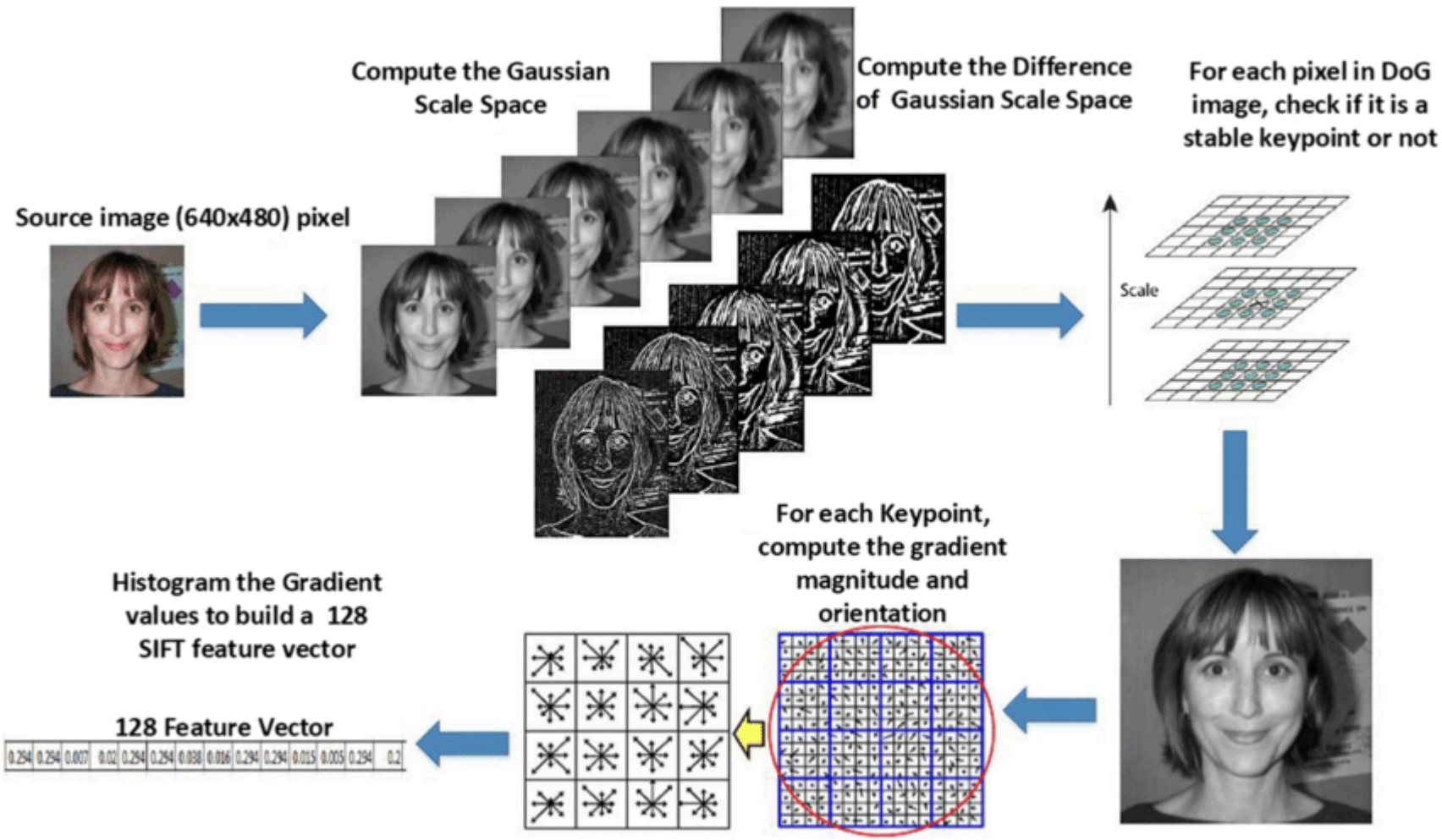
- Fit a 3D quadratic function to refine:
  - Position
  - Scale
- Remove:
  - Low-contrast points
  - Edge-like responses (Hessian ratio test)

Orientation Assignment

- Compute gradient magnitudes and orientations around the keypoint
- Build a histogram (36 bins)
- Dominant peak = reference orientation
- Ensures <u>rotation invariance</u>

Part II — Feature Descriptor

Overview


Source image (640x480) pixel
Compute the Gaussian Scale Space
Compute the Difference of Gaussian Scale Space
For each pixel in DoG image, check if it is a stable keypoint or not
Scale
For each Keypoint, compute the gradient magnitude and orientation
Histogram the Gradient values to build a 128 SIFT feature vector
128 Feature Vector

Extract Local Patch

- Take a 16×16 window aligned with the keypoint's orientation
- Weighted by Gaussian to reduce boundary effects
- Divide into 4×4 cells

Orientation Histograms per Cell

- Each cell produces an 8-bin histogram
- Descriptor length:
  - 4×4 cells $\times$ 8 bins = **128 dimensions**

Normalization

- L2-normalization to reduce effect of illumination
- Threshold large values and renormalize
- Achieves robustness to:
  - Contrast changes
  - Non-linear illumination variations

Why SIFT Works Well

- Gradient-based: captures local structure
- Spatial distribution preserved via cell grid
- Invariance properties:
  - Scale
  - Rotation
  - Illumination

Uses of SIFT

- Widely used in:
  - SLAM
  - Structure-from-Motion
  - Aerial mapping

Limitations & Alternatives

Limitations

- Computationally expensive
- Not ideal for realtime onboard drone systems
- Historically patent-restricted (expired 2020)

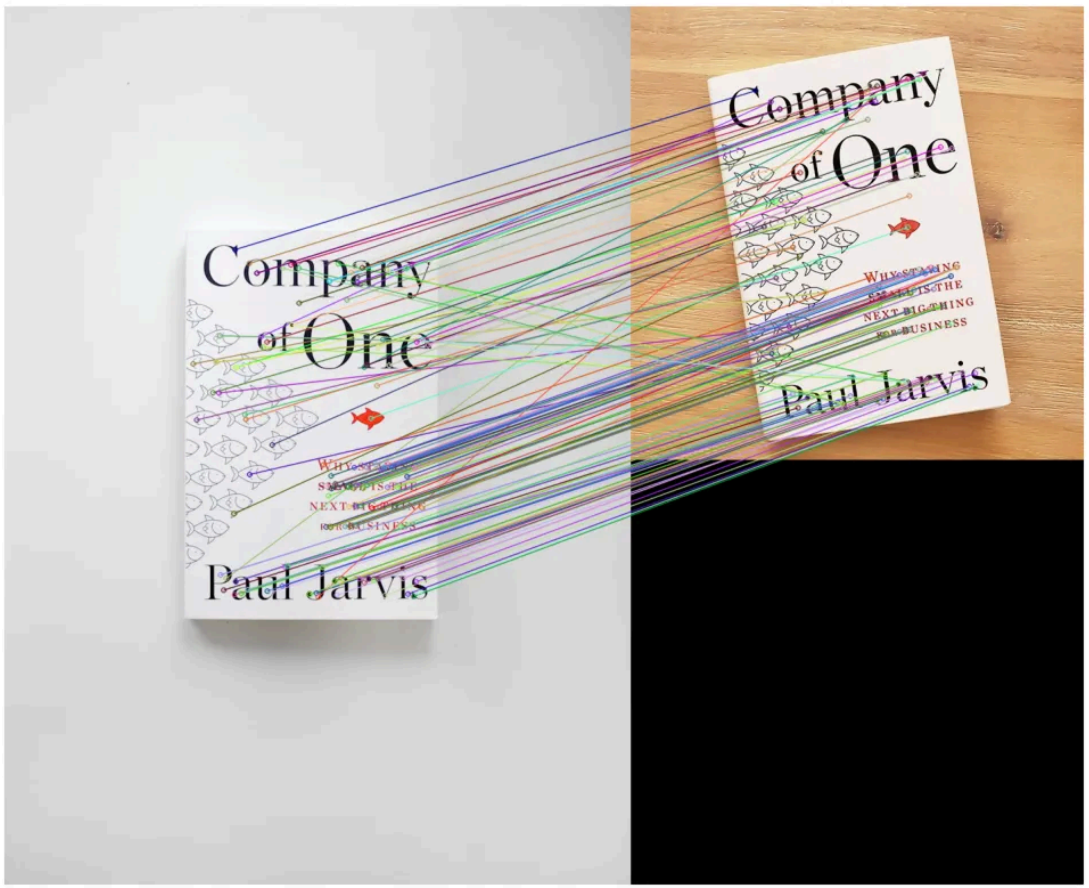
Alternatives

- ORB (fast, binary)
- AKAZE/SURF
- Deep-learning features (SuperPoint, D2-Net)

Applications in Drone Perception

- Stitching aerial mosaics
- Landmark-based localization
- Feature tracking across frames
- 3D reconstruction from drone surveys
- Change detection over large-scale environments

Example: SIFT Matching



The matching process

Filtering matches

Lowe filtering

- The best match should be significantly better than the next best match

RANSAC filtering based on

- Fundamental matrix
- Essential matrix

Summary

Detector:

- Scale-space extrema
- Accurate keypoint localization
- Orientation assignment

Descriptor:

- 128-dim gradient histograms
- Robust to scale, rotation, illumination

Still a foundational method in computer vision

Estimating motion

Motion relative to what?

- Initial frame
- Previous frame
- General motion in the video

Different approaches

- Optical flow
- Tracking of key points

Motion relative to what?

Initial frame

- We don’t expect the camera to move much when observing traffic

Different approaches

- Tracking of key points
  - SIFT
  - AKAZE
  - ORB

Video stabilizer code

See one of the exercises for today

Outro

Course overview

2026-02-01

- Python and 2D image processing
- Camera in space
- Optimization
- Fiducial markers

2026-03-01

- Camera calibration mini project
- Feature detectors and video stabilization
- Motion detection mini project
- Visual odometry

2026-04-01

- Visual odometry mini project
- Orthomosaics
- Pumpkin counting mini project

2026-05-01

- Bag of Visual Words / AI overview

SDU

Quote

“If it doesn’t work, it doesn’t matter how fast it doesn’t work.”

— Mich Ravera